Relational Databases and SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Introduction

What is a Database?

A structured set of data held in a computer, especially one that is accessible in various ways.

- **Purpose:** To store, retrieve, and manage large amounts of information.
- **Evolution:**
  - **Flat files:** CSV/Text (Fast but limited).
  - **Hierarchical:** Tree-like structure (IBM IMS).
  - **Relational:** Tables with relationships (The standard since the 80s).
  - **NoSQL:** Flexible schemas for big data (MongoDB, Redis).

What is an RDBMS?

A **Relational Database Management System (RDBMS)** is the software layer that manages relational databases.

- It provides an interface between users/applications and the data.
- Examples: **SQLite, PostgreSQL, MySQL, Oracle, Microsoft SQL Server**.
- It enforces the **Relational Model**, data integrity, and handles multi-user access.

RDBMS vs DB vs Relational Model

- **Relational Model:** The theoretical framework (Codd, 1970) using tables (relations).
- **Database (DB):** The actual collection of data (e.g., the `university.db` file).
- **RDBMS:** The engine/software (e.g., `sqlite3` or `PostgreSQL`) that manages the DB using the Relational Model.

Concept	Nature	Responsibility
Relational Model	Theory	Defines structures (Tables, Keys).
Database (DB)	Data	The stored values and metadata.
RDBMS	Software	Execution, Security, Integrity, ACID.

Advantages of RDBMS

ACID Properties I

An RDBMS guarantees data integrity through ACID:

- **Atomicity:** “All or nothing.” If any part of a transaction fails, the entire transaction is rolled back.
- **Consistency:** The database must transition from one valid state to another, following all rules (constraints).

ACID Properties II

- **Isolation:** Simultaneous transactions do not interfere; they appear to run sequentially.
- **Durability:** Once committed, data is permanent, surviving system crashes or power failures.

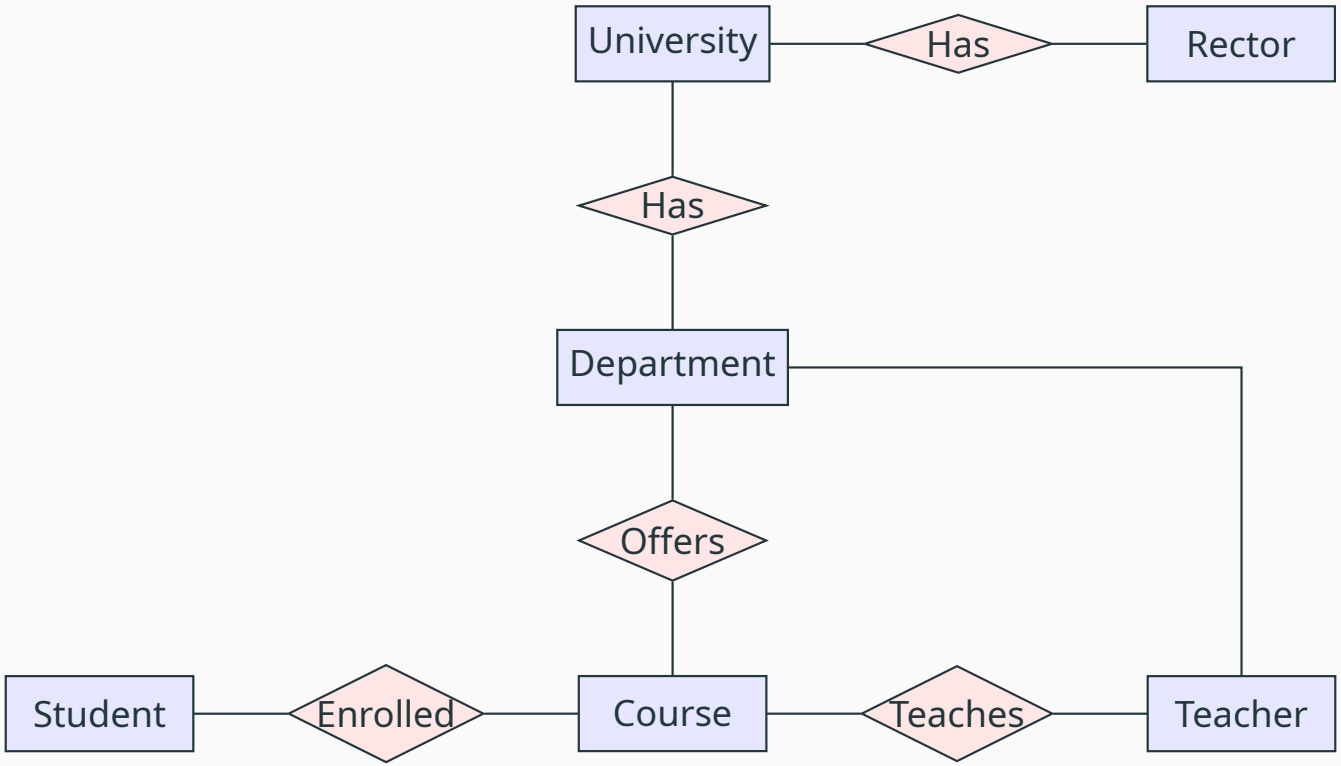
The Relational Model

University Scenario

To understand these concepts, we will use a **University Database** scenario:

- **University:** The institution.
- **Rector:** The head of the university (1:1 relationship).
- **Department:** Organizational units (Engineering, Arts).
- **Teacher:** Staff assigned to departments.
- **Course:** Subjects offered by departments.
- **Student:** Individuals enrolled in courses (N:M relationship).

Entity-Relationship (ER) Diagram

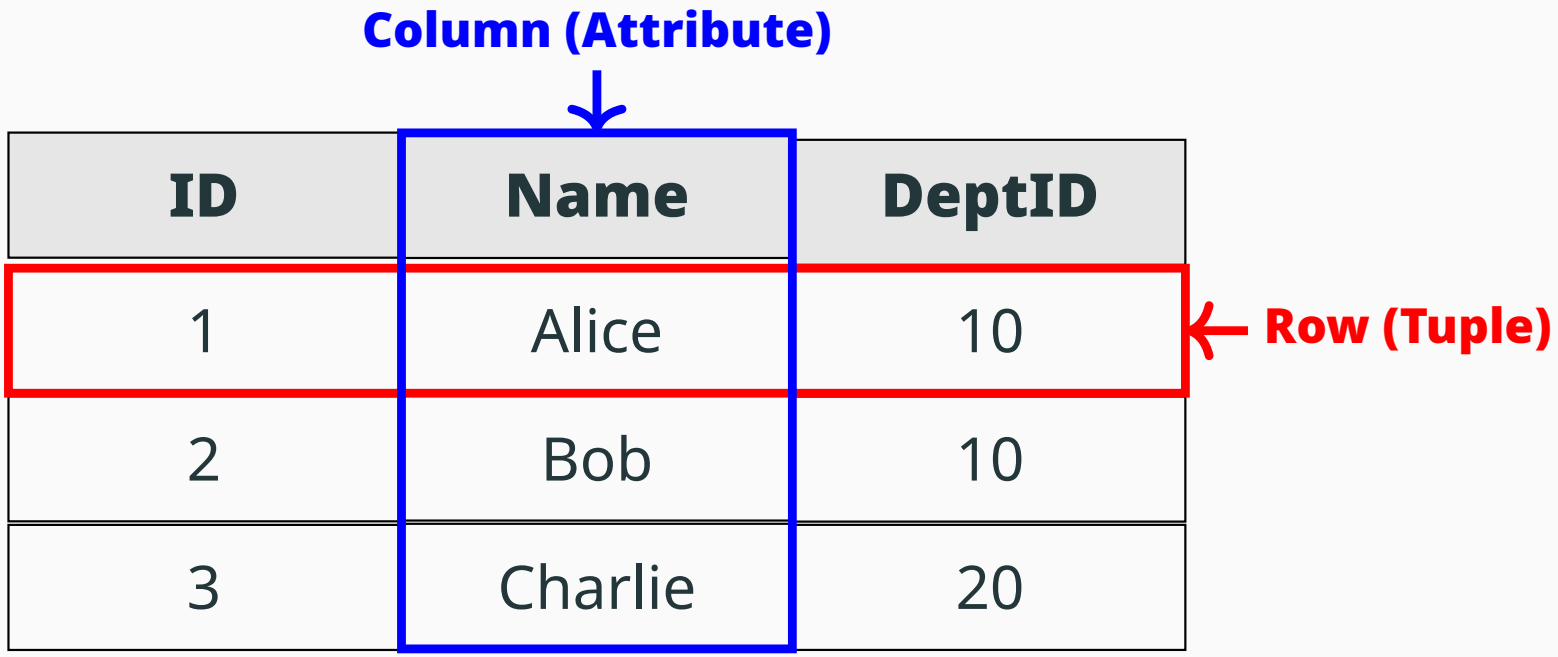


Core Concepts I

Proposed by Edgar F. Codd in 1970, it organizes data into one or more tables.

- **Table (Relation):** A collection of data elements organized in rows and columns.
- **Row (Tuple/Record):** Represents a single, unique item in the table.
- **Column (Attribute/Field):** Represents a specific property of the items.

Core Concepts II



Core Concepts III: Keys

- **Primary Key (PK):** A unique identifier for every row.
  - **Natural Key:** Something that already exists (e.g., Student ID).
  - **Surrogate Key:** An artificial ID (e.g., `id = 1, 2, 3`).
- **Foreign Key (FK):** A column that creates a link between two tables.
- **Composite Key:** A primary key composed of two or more columns.

Database Relationships I: 1:1

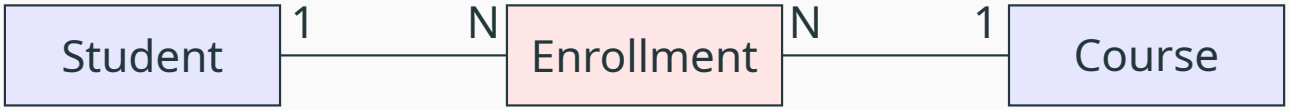
- **One-to-One (1:1):** One entity is related to exactly one other.
- Example: A **University** has one **Rector**.

Database Relationships II: 1:N

- **One-to-Many (1:N):** One entity can be related to many of another.
- Example: One **Department** has many **Teachers**.

Database Relationships III: N:M

- **Many-to-Many (N:M):** Many of one entity relate to many of another.
- Example: Many **Students** enroll in many **Courses**.



Junction Table

SQL: Structured Query Language

SQL Sublanguages (6 Models) I

SQL is divided into several sublanguages for different purposes:

1. **DQL (Data Query Language):** To retrieve data.
   - `SELECT`
2. **DML (Data Manipulation Language):** To modify data.
   - `INSERT`, `UPDATE`, `DELETE`
3. **DDL (Data Definition Language):** To define schema.
   - `CREATE`, `ALTER`, `DROP`, `TRUNCATE`

SQL Sublanguages (6 Models) II

4. **DCL (Data Control Language):** To control access/permissions.
   - `GRANT`, `REVOKE`
5. **TCL (Transaction Control Language):** To manage transactions.
   - `COMMIT`, `ROLLBACK`, `SAVEPOINT`
6. **Administrative/Metadata:** To manage the system or inspect metadata.
   - `DESCRIBE`, `EXPLAIN`, `PRAGMA` (SQLite specific)

SQL Data Types

Common types used in RDBMS (Standard and SQLite):

- **INT / INTEGER:** Whole numbers.
- **VARCHAR(n) / TEXT:** Character strings.
- **REAL / FLOAT / DOUBLE:** Decimal numbers.
- **DATE / DATETIME:** Temporal values.
- **BOOLEAN:** True/False (often stored as 0/1 in SQLite).
- **BLOB:** Binary data (images, files).

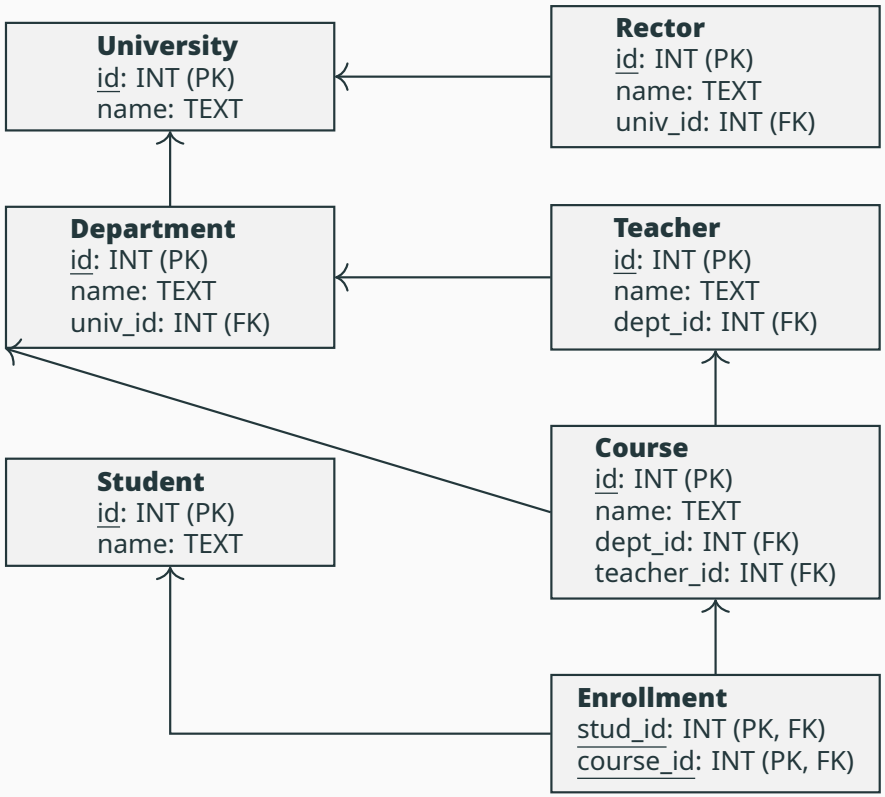
SQL Aggregate Functions

Functions that perform a calculation on a set of values:

- **COUNT():** Returns the number of rows.
- **SUM():** Returns the total sum of a numeric column.
- **AVG():** Returns the average value.
- **MIN() / MAX():** Returns the smallest/largest value.

```sql
SELECT COUNT(*) FROM Student;
SELECT AVG(id) FROM Course; -- Example calculation
```

Relational Schema Diagram



End-to-End SQL: DDL (Schema)

DDL: Creating Tables I

```sql
CREATE TABLE University (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL
);

CREATE TABLE Rector (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
    univ_id INTEGER UNIQUE, FOREIGN KEY (univ_id) REFERENCES University
);
```

DDL: Creating Tables II

```sql
CREATE TABLE Department (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
    univ_id INTEGER, FOREIGN KEY (univ_id) REFERENCES University(id)
);

CREATE TABLE Teacher (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
    dept_id INTEGER, FOREIGN KEY (dept_id) REFERENCES Department(id)
);
```

DDL: Creating Tables III

```
CREATE TABLE Course (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
    dept_id INTEGER, teacher_id INTEGER,
    FOREIGN KEY (dept_id) REFERENCES Department(id),
    FOREIGN KEY (teacher_id) REFERENCES Teacher(id)
);

CREATE TABLE Student (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL
);
```

DDL: Creating Tables IV

```sql
CREATE TABLE Enrollment (
    stud_id INTEGER, course_id INTEGER,
    PRIMARY KEY (stud_id, course_id),
    FOREIGN KEY (stud_id) REFERENCES Student(id),
    FOREIGN KEY (course_id) REFERENCES Course(id)
);
```

End-to-End SQL: DML (Data)

DML: Inserting Data I

```sql
INSERT INTO University (name) VALUES ('Univ. Aveiro');

INSERT INTO Rector (name, univ_id) VALUES ('Prof. Paulo', 1);

INSERT INTO Department (name, univ_id) VALUES ('DETI', 1), ('DMat', 1);

INSERT INTO Teacher (name, dept_id) VALUES ('Dr. Smith', 1), ('Dr. Tayl
```

DML: Inserting Data II

```sql
INSERT INTO Course (name, dept_id, teacher_id)
VALUES ('Relational Databases', 1, 1), ('Linear Algebra', 2, 2);

INSERT INTO Student (name) VALUES ('Alice'), ('Bob');

INSERT INTO Enrollment (stud_id, course_id)
VALUES (1, 1), (1, 2), (2, 1);
```

DML: Updates and Deletions

```sql
-- Modifying existing records
UPDATE Teacher SET name = 'Prof. Smith' WHERE id = 1;

-- Removing records
DELETE FROM Enrollment WHERE stud_id = 2 AND course_id = 2;
```

End-to-End SQL: DQL (Queries)

DQL: Basic Queries and Filtering

```sql
-- Select specific columns with an alias
SELECT name AS StudentName FROM Student;

-- Filtering with WHERE
SELECT * FROM Course WHERE dept_id = 1;

-- Aggregate Functions
SELECT COUNT(*) FROM Enrollment WHERE course_id = 1;
```

DQL: Complex Query (Joins)

```sql
SELECT s.name AS Student, c.name AS Course, t.name AS Teacher,
       d.name AS Dept, u.name AS Univ, r.name AS Rector
FROM Student s
JOIN Enrollment e ON s.id = e.stud_id
JOIN Course c ON e.course_id = c.id
JOIN Teacher t ON c.teacher_id = t.id
JOIN Department d ON t.dept_id = d.id
JOIN University u ON d.univ_id = u.id
JOIN Rector r ON u.id = r.univ_id;
```

Expected Results I

Student	Course	Teacher
Alice	Relational Databases	Prof. Smith
Alice	Linear Algebra	Dr. Taylor
Bob	Relational Databases	Prof. Smith

Expected Results II

Dept	Univ	Rector
DETI	Univ. Aveiro	Prof. Paulo
DMat	Univ. Aveiro	Prof. Paulo
DETI	Univ. Aveiro	Prof. Paulo

Other SQL Models

DCL: Data Control Language

Manages access and permissions (Conceptual in some RDBMS):

```
-- Grant read-only access to a teacher
GRANT SELECT ON Student TO 'teacher_user';

-- Revoke delete permissions from students
REVOKE DELETE ON Course FROM 'student_role';
```

TCL: Transaction Control Language

Ensures ACID properties by grouping statements:

```
BEGIN TRANSACTION;
INSERT INTO Student (name) VALUES ('Eve');
-- If system fails here, Eve is NOT added permanently
COMMIT; -- Saves changes

-- Or revert if an error is detected:
-- ROLLBACK;
```

Administrative & Metadata

Commands to manage the database engine itself:

```sql
-- MySQL / PostgreSQL:
EXPLAIN SELECT * FROM Student;
DESCRIBE Course;

-- SQLite:
PRAGMA foreign_keys = ON;
PRAGMA table_info('Student');
```

Summary

Summary

- **RDBMS:** Software layer (SQLite) enforcing the **Relational Model**.
- **ACID:** Foundational advantages (Atomicity, Consistency, Isolation, Durability).
- **SQL Models:** DQL, DML, DDL, DCL, TCL, and Administrative.
- **Structure:** PK/FK relationships visualized via ER and Cardinality diagrams.